

Learned Embedding Cost cho Cell-Ancestry Inference

Tổng hợp nghiên cứu, insight và research question — hướng đến CVPR Bio-Image track

2026-09-30

Tóm tắt

Một learned metric-embedding cost — thay cho cost thủ công (khoảng cách pixel, OT thủ công) mà mọi baseline trước đây dùng — kết hợp với decode bằng balanced entropic optimal transport (OT), lần đầu tiên thắng hoặc hòa mọi baseline (LAP-capacitated, PyUAT ILP) ở **cả ba** chế độ thừa dữ liệu (ends, s16, s32) cùng lúc trên dataset gốc **One-in-a-Million (OIAM)**, thay vì phải đánh đổi giữa chúng như trước. Kết quả chốt trên OIAM: `embed_v3_ot_e0.5_tinf` đạt ends=0.400, s16=0.744, s32=0.668, so với PyUAT (0.313/0.151, 0.802, 0.589) và LAP2 cũ (0.373, 0.706, 0.577).

Sau đó, thí nghiệm mở rộng sang 3 dataset CTC khác (SIM+, MuSC, HSC) cho một bức tranh **phức tạp hơn nhiều** so với kỳ vọng ban đầu: zero-shot transfer (model train trên OIAM, không retrain) sụp mạnh trên dữ liệu mô phỏng (SIM+), chỉ ngang bằng baseline trên dữ liệu thật (MuSC/HSC). Domain-adaptation (train lại trên 1 sequence, test trên sequence còn lại cùng dataset) cải thiện rõ rệt trên MuSC (thắng baseline ở cả 3 schedule) nhưng lại **làm tệ hơn** trên HSC (thua baseline ở cả 3 schedule). Kết luận trung thực: pattern “một cost học được thắng mọi nơi” **chỉ được xác nhận trên OIAM**; claim tổng quát hóa xuyên dataset hiện chưa đủ bằng chứng để đưa vào paper như một kết quả chính — đây, cùng với việc chưa so sánh với Trackastra (SOTA công bố sẵn trên chính OIAM, chính trực robustness này), là hai rào cản lớn nhất cho CVPR, quan trọng hơn nhiều so với khoảng cách s16 còn lại trên OIAM.

1. Bài toán & Động lực

Benchmark: ancestor-chaining trên time-lapse imaging thừa mẫu — với mỗi cell tại khung hình quan sát cuối b , dự đoán ancestor của nó tại khung hình neo a , đánh giá qua `score_window` (macro-average theo window trong movie, rồi theo movie), trên 3 mức độ thừa: ends (chỉ 2 khung hình, cách xa nhau), s16, s32 (khoảng cách trung bình).

Vì sao phải quan sát thừa thay vì đầy đủ: hai ràng buộc vật lý của kính hiển vi sống (live-cell imaging). Một, phototoxicity/photobleaching — chiếu sáng liên tục để chụp dày khung hình sẽ làm tổn thương tế bào sống hoặc làm phai tín hiệu huỳnh quang, đặc biệt với thí nghiệm chạy dài. Hai, đánh đổi thông lượng kính hiển vi — số buồng nuôi cấy có thể theo dõi đồng thời và tần suất chụp mỗi buồng tỉ lệ nghịch nhau; các nghiên cứu đa buồng/đa điều kiện (như OIAM) buộc phải chụp thừa hơn trên mỗi buồng để theo dõi được nhiều buồng cùng lúc. Sparse sampling không phải một lựa chọn thiết kế benchmark tùy tiện — nó là ràng buộc thực nghiệm không thể tránh khỏi.

Baseline có sẵn trước khi bắt đầu hướng này:

- Chaining bằng khoảng cách pixel thô + LAP (Hungarian có capacity) hoặc entropic OT thủ công (`cellanc/transport.py`) — cost hand-set, không học được đặc trưng hình dạng/chuyển động.
- **PyUAT** (`experiments/pyuat_run.py`): ILP tracker khai thác shape + motion/division likelihood đồng thời trên nhiều khung hình. Thắng áp đảo baseline LAP/OT ở s16/s32 (~0.6–0.8 so với ~0.45–0.55), nhưng **collapse tại ends** (2 khung hình) vì thiếu quan sát để fit likelihood — rơi xuống dưới cả nearest-neighbour.

Khoảng trống: không có phương pháp nào thắng ở cả hai đầu (ends lần moderate sparsity) cùng lúc. Ý tưởng cốt lõi của hướng nghiên cứu này: thay cost thủ công bằng một **learned embedding cost** duy nhất, huấn luyện trên curriculum nhiều mức Δt (1 đến 132 phút) để không cần chuyển đổi phương pháp theo độ thưa dữ liệu.

2. Phương pháp

Kiến trúc Embed (cellanc/embed.py):

```
phi(features) -> R^16          embedding mỗi cell (MLP 2 lớp ẩn)
g(phi(cell), Δt)              residual motion predictor: embedding dự đoán Δt phút sau
cost(i@t, j@t+Δt) = ||g(phi(i),Δt) - phi(j)||^2 * exp(-log_tau)
```

Huấn luyện (experiments/train_embed.py): listwise softmax cross-entropy — mỗi cell target so với mọi ứng viên ancestor trong cùng khung nguồn — trên curriculum gap theo từng dataset (OIAM: [1,2,4,8,16,32,64,132] phút=khung hình). Việc feed Δt tường minh (tính bằng phút thật, time_step_min, không phải số khung hình thô) và huấn luyện trên mọi mức gap là điểm khắc phục trực tiếp nhược điểm “collapse tại ends” của PyUAT.

Feature set ($N_FEATURES$, tiến hóa qua các version, xem mục 3): vị trí chuẩn hóa, trục hình dạng + log kích thước, mật độ cục bộ ($k=5$ trung bình), vận tốc (từ khung quan sát liền trước cùng track id, có dropout khi huấn luyện để robust với cell mới xuất hiện), khoảng cách hàng xóm gần nhất (cờ báo chia tách tế bào). Với dataset không có oracle mask đầy đủ (CTC gold SEG thừa), các đặc trưng hình dạng fallback về NaN $\rightarrow$ 0 một cách graceful.

Chỉ dùng oracle feature: hình dạng/vị trí lấy từ oracle mask qua gt_mapping — đây là phát triển phương pháp trên nền oracle, chưa phải pipeline detector thật (xem mục Hạn chế).

3. Thí nghiệm: Feature Ablation (trên OIAM)

Version	$N_FEATURES$	Thêm gì	Insight
v0	7	vị trí, trục, log shape, log spacing	Go/no-go ban đầu cho learned cost
v1	7	(giống v0, huấn luyện lâu hơn, t-stride=3)	Cải thiện nhỏ — huấn luyện lâu hơn không phải nút thắt chính
v2	10	+ vận tốc (vy, vx, has_velocity)	Cải thiện lớn nhất session: s16 từ 0.595 $\rightarrow$ 0.707
v3	11	+ log_nearest_dist (cờ chia tách)	Cải thiện biên: s32 chỉ +0.014

Vì sao vận tốc là đòn bẩy lớn nhất: một chẩn đoán trực tiếp (so sánh độ chính xác single-hop) cho thấy embedding không có vận tốc thiếu đúng loại tín hiệu mà motion-likelihood của PyUAT có — v1 chỉ thắng PyUAT ở single-hop khi PyUAT phải suy luận qua chuỗi 8-hop. Thêm vận tốc cho phép model học “quán tính chuyển động” của mỗi tế bào mà không cần fit ILP nhiều khung hình.

Giả thuyết “compounding error” đã bị bác bỏ: ban đầu nghĩ điểm yếu s16/s32 đến từ lỗi tích lũy qua nhiều bước chaining tham lam ($0.708^8 \approx 0.057$ dự đoán quá bi quan so với $\sim 0.58\text{--}0.60$ thực tế đo được). Nguyên nhân thật là chất lượng cost per-hop yếu ở Δt trung bình, không phải compounding — dẫn trực tiếp đến việc thêm vận tốc.

4. Thí nghiệm: Decode Strategy (trên OIAM)

Giữ nguyên checkpoint đã huấn luyện, chỉ đổi cách decode cost matrix thành lineage:

Decode	Cơ chế	Kết quả
nearest lap (cap=2/4/soft)	argmin mỗi hop, không capacity Hungarian có capacity mỗi hop (row-replication)	Baseline Chứng minh được là tối ưu toàn cục cho mục tiêu tổng chi phí theo chuỗi, với ràng buộc capacity chỉ theo từng hop
soft	Belief propagation qua column-softmax	Luôn kém hơn lap2 ở mọi nhiệt độ (thiếu ràng buộc capacity cứng, temperature càng cao càng làm mờ kết quả)
dp (skip-connection)	Shortest-path DP trên MỌI cặp khung hình, không chỉ liền kề	Kém hẳn nearest (s16: 0.335 so với 0.656) — cost ở các Δt khác nhau không được hiệu chỉnh cùng thang đo, DP bị “lừa” bởi các đường tắt xa có chi phí thấp giả tạo
ot	Balanced entropic Sinkhorn OT (mass = diện tích tế bào), eps=0.5, tau=None	Đột phá trên OIAM : thắng lap2/lap4 ở MỌI schedule đồng thời, thắng PyUAT tại s32

Chứng minh toán học quan trọng (xác nhận qua kiểm tra độc lập): với một chuỗi hop mà ràng buộc capacity chỉ áp dụng theo từng hop và mục tiêu là tổng chi phí cộng dồn, giải LAP tối ưu độc lập từng hop **đã là tối ưu toàn cục** — không có bộ giải “global flow” nào dùng cùng lượng thông tin per-hop có thể cải thiện thêm.

Vì sao OT thắng lap dù cùng dùng per-hop info: OT-balanced ép ràng buộc capacity qua **bảo toàn khối lượng thực sự** (mỗi target phải được giải thích đầy đủ bởi nguồn, trọng số theo diện tích tế bào), không có tham số cap tùy chỉnh phải chọn tay theo từng schedule như LAP.

5. Kết quả cuối cùng trên OIAM

Macro-average trên M3+M4 (embed) / M0–M4 (PyUAT), 3 schedule ends / s16 / s32:

Method	ends	s16	s32
embed_v3_lap2 (baseline learned cost)	0.373	0.706	0.577
embed_v3_ot_e0.5_tinf (chốt)	0.400	0.744	0.668
pyuat_FO+G+O+DD (ILP đa likelihood)	0.313 / 0.151*	0.802	0.589

* PyUAT có hai biến thể tại ends: cấu hình chính (0.313) và biến thể nn (0.151) — cả hai đều thấp hơn hẳn learned embedding, minh chứng cho việc PyUAT collapse khi thiếu quan sát.

Đây là cấu hình đầu tiên trong toàn bộ nỗ lực nghiên cứu mà embedding thắng hoặc hòa ở mọi chế độ thừa dữ liệu cùng lúc trên OIAM — trước đó luôn là đánh đổi (embedding tốt ở ends, PyUAT tốt ở s16/s32). **Toàn bộ mục 3–5 là kết quả in-domain (train và test trên cùng OIAM)** — phần tiếp theo kiểm tra liệu pattern này có sống sót khi rời khỏi OIAM hay không.

6. Thí nghiệm mở rộng: Multi-dataset Transfer & Domain Adaptation

Hạ tầng pipeline (`main.py fetch/index/build/baseline`) đã có sẵn cho 4 dataset CTC khác (SIM+, HSC, MuSC, HeLa) từ trước, chỉ cần tổng quát hóa `eval_embed.py/train_embed.py` (vốn hard-code cho OIAM) để chạy được trên chúng. Trong lúc tổng quát hóa, phát hiện và sửa một **bug đơn vị Δt** trong `train_embed.py`: huấn luyện trước đó dùng số khung hình thô làm Δt , trong khi eval dùng phút thật ($\Delta t \times \text{time_step_min}$) — vô hình trên OIAM vì `time_step_min=1.0` (khung hình trùng phút), nhưng sẽ âm thầm làm sai lệch mọi model train trên dataset có `time_step_min != 1` (MuSC/HSC=5 phút/khung hình). Đã sửa trước khi bug này ảnh hưởng tới kết quả domain-adaptation bên dưới.

6.1 Zero-shot transfer (train trên OIAM, không retrain, eval trực tiếp)

Decode ranking nội tại $\text{OT} \geq \text{lap2} \geq \text{nearest}$ **giữ nguyên** trên mọi dataset — đây là phát hiện ổn định nhất trong toàn bộ thí nghiệm mở rộng. Nhưng so với baseline vị trí đơn giản (`chained_lap/nearest`):

Dataset	embed OT (ends/s16/s32)	Best baseline (ends/s16/s32)	Kết luận
SIM+ (mô phỏng, H=57)	0.52 / 0.39 / 0.42	0.88 / 0.97 / 0.95	Sụp nặng — thua xa baseline vị trí đơn giản
MuSC (thật, H=110, macro cả 2 seq)	≈ 0.19 / 0.24 / 0.23	≈ 0.17 / 0.21 / 0.20	Ngang bằng/nhỉnh hơn chút
HSC (thật, H=239, macro cả 2 seq)	≈ 0.16 / 0.13 / 0.19	≈ 0.19 / 0.21 / 0.19	Ngang bằng, không sụp nhưng không thắng

Insight: chất lượng transfer phụ thuộc mạnh vào độ tương đồng domain với chuyển động vi khuẩn thật của OIAM — dữ liệu mô phỏng (SIM+) thất bại nặng, chuyển động sinh học thật (MuSC/HSC) transfer “tạm ổn” nhưng không ấn tượng.

6.2 Domain-adaptation (train lại trên 1 sequence cùng dataset, test trên sequence còn lại — held-out, không leak)

Dùng đúng split đã có sẵn trong `main.py` (`domain_adapt` để train, `domain_test` để eval), train 60 epoch mỗi dataset. Kết quả tách riêng cho sequence test (không macro-average gộp với sequence train như phần 6.1, để so sánh công bằng theo đúng sequence được eval):

MuSC-02 (H=110) — thắng toàn diện:

Method	ends	s16	s32
Best baseline	0.165	0.232	0.196
Zero-shot embed + OT	0.166	0.203	0.176
Domain-adapted embed + OT	0.206	0.246	0.212

HSC-02 (H=239) — thua toàn diện:

Method	ends	s16	s32
Best baseline	0.173	0.179	0.204
Zero-shot embed + OT	0.145	0.168	0.148
Domain-adapted embed + OT	0.112	0.126	0.177

Lưu ý về tính công bằng so sánh: baseline (LAP/nearest) không cần dữ liệu huấn luyện và không hề dùng nhãn của sequence “-01” — trong khi embed domain-adapted đã được huấn luyện thêm trên chính

nhân đó. Đây là một sự bất đối xứng thông tin, không phải so sánh ngang hàng đơn thuần. Diễn giải đúng: “khi có một lượng nhỏ nhân domain đích để calibrate, embed cost học được có thể tận dụng được (MuSC), nhưng không phải luôn luôn (HSC) — trong khi LAP/nearest cost cố định không có cơ chế nào để tận dụng dù được cho thêm dữ liệu đó.” Một thí nghiệm cần làm để so sánh công bằng hơn: calibrate cả baseline (vd. grid-search cap/threshold của LAP) trên cùng split domain_adapt rồi mới so sánh.

Insight cốt lõi của phần 6: domain-adaptation **không phải giải pháp chắc chắn** — cải thiện rõ trên MuSC, làm tệ hơn trên HSC (train_acc trên tập train của HSC cũng thấp hơn hẳn: 0.737 so với 0.835 của MuSC, gợi ý model học kém ổn định hơn trên domain này, có thể vì H=239 lớn hơn nhiều làm tín hiệu chuyển động khó học). Bằng chứng hiện tại: 1/2 dataset domain-adapt thành công — chưa đủ để khẳng định một quy luật tổng quát.

7. Insight chính

1. **Vận tốc là tín hiệu đơn lẻ có đòn bẩy cao nhất** trên OIAM — không phải kiến trúc phức tạp hơn hay huấn luyện lâu hơn.
2. **Compounding error không phải nguyên nhân** của điểm yếu moderate-sparsity trên OIAM — chất lượng cost per-hop mới là gốc rễ.
3. **Giải per-hop capacitated LAP đã tối ưu toàn cục** cho mục tiêu chuỗi cộng dồn — hướng “global solver thông minh hơn” dùng cùng thông tin là ngõ cụt đã được chứng minh.
4. **Ràng buộc capacity “mềm” cần cơ chế bảo toàn khối lượng thật (OT), không chỉ softmax** — soft belief propagation thất bại vì thiếu ràng buộc cứng.
5. **Decode ranking $OT \geq LAP \geq nearest$ là bất biến xuyên dataset** — đây là phát hiện *duy nhất* trong toàn bộ nỗ lực đã sống sót qua mọi điều kiện thử nghiệm (in-domain, zero-shot, domain-adapted, 4 dataset khác nhau). Có thể là claim đáng tin cậy nhất cho paper, đáng tin cậy hơn nhiều so với claim về bản thân learned cost.
6. **Learned cost transfer xuyên domain không nhất quán:** thắng đậm trên OIAM in-domain, sụp trên dữ liệu mô phỏng, tạm ổn zero-shot trên dữ liệu thật khác, và domain-adaptation chỉ sửa được vấn đề này ở 1/2 trường hợp thử. Đây là giới hạn thật, không phải vấn đề kỹ thuật chưa giải quyết.
7. **So sánh “domain-adapted embed vs. non-adapted baseline” mang bất đối xứng thông tin** cần được gắn nhãn rõ ràng hoặc bổ sung baseline đã calibrate để công bằng.

8. Hạn chế

- **Kết quả “thắng mọi nơi” chỉ được xác nhận trên OIAM** (in-domain). Trên 3 dataset khác: zero-shot sụp nặng (SIM+) hoặc ngang bằng (MuSC/HSC); domain-adaptation thắng 1/2 (MuSC), thua 1/2 (HSC).
- **Oracle mask/feature:** shape/vị trí lấy từ ground-truth mapping, không phải từ pipeline detector/segmentation thật, trên toàn bộ dataset.
- **s16 vẫn thua PyUAT trên OIAM** ~6 điểm (0.744 vs 0.802).
- **Chưa có so sánh với Trackastra** — phát hiện quan trọng: Trackastra là **SOTA công bố sẵn trên chính OIAM, chính trực robustness-theo-imaging-interval này** (paper gốc giới thiệu OIAM, arXiv 2411.00552, Table 3: Trackastra LNK=0.50/0.42, DIV=0.45/0.32 tại RM@0.8/0.9, vượt xa LAP/Distance baseline). Thiếu so sánh này là rủi ro lớn nhất cho reviewer, lớn hơn cả gap s16 hay vấn đề multi-dataset.
- **So sánh domain-adapted vs baseline không cùng lượng thông tin** (baseline không được calibrate trên dữ liệu domain đích) — cần thí nghiệm bổ sung để công bằng.
- **Chưa đo chi phí tính toán** (wall-clock/FLOPs) embed+OT so với PyUAT ILP hay Trackastra.

9. Research Question cuối cùng

Một learned per-cell embedding cost, decode bằng balanced entropic optimal transport, thắng áp đảo baseline LAP cổ điển và cạnh tranh với PyUAT trên một dataset thừa mẫu thực tế (OIAM) — nhưng liệu đóng góp cốt lõi của paper nên là **claim hẹp và chắc chắn** (“trên OIAM, cost học

được + OT decode vượt trội LAP ở mọi độ thưa”), hay **claim rộng về tổng quát hóa xuyên dataset** mà bằng chứng hiện tại (1 thắng/1 thua trong domain-adaptation, sụp trong zero-shot trên dữ liệu mô phỏng) chưa đủ mạnh để bảo vệ trước phản biện?

Bằng chứng hiện có nghiêng về phương án claim hẹp: coi phần multi-dataset là một mục “Generalization Analysis” trung thực (bao gồm cả kết quả âm) thay vì một kết quả chính, và tập trung làm chắc kết quả OIAM (đặc biệt là so với Trackastra) trước khi mở rộng thêm.

10. Next Actions

Ưu tiên	Hành động	Vì sao
Cao nhất	So sánh với Trackastra trên chính benchmark của dự án	Đã xác nhận là SOTA công bố sẵn trên đúng dataset/đúng trực robustness — thiếu nó, mọi số liệu khác sẽ bị nghi ngờ ngay lập tức bởi reviewer
Cao	Viết claim chính giới hạn ở OIAM in-domain; đưa multi-dataset thành “Generalization Analysis” trung thực (thắng + thua)	Bằng chứng hiện tại (1/2 domain-adapt thành công, sụp zero-shot trên SIM+) chưa đủ cho claim tổng quát hóa rộng
Trung bình	Calibrate baseline (LAP threshold/cap) trên split <code>domain_adapt</code> để so sánh công bằng với embed domain-adapted	Sửa bất đối xứng thông tin đã phát hiện ở mục 6.2/8
Trung bình	Thay oracle mask bằng detector/segmentation thật	Reviewer Bio-Image sẽ hỏi ngay; hiện tại chỉ là method development
Trung bình	Đo chi phí tính toán (wall-clock/FLOPs) embed+OT vs PyUAT ILP / Trackastra mỗi window	Biến câu chuyện thành Pareto frontier rõ ràng, không chỉ accuracy
Thấp	Tìm hiểu vì sao domain-adapt thất bại trên HSC cụ thể (train_acc thấp hơn MuSC)	Có thể cải thiện được (nhiều epoch hơn, feature khác) nhưng không chặn các bước trên
Thấp	Thử thêm feature appearance/CNN để thu hẹp gap s16 trên OIAM	Lợi ích biên giảm dần đã thấy rõ (v2 → v3 chỉ +0.014); ưu tiên thấp hơn các mục trên